Milvus blog for repurpose

Original articles

从25年9月1日起发布的所有Milvus 官网blog

表格

	Title & Topic	Publish URL
1	⚠️Milvus Week: Introducing the Mil...	https://milvus.io/blog/milvus-ngram-index-faster-k...
2	⚠️Milvus Week: Stop Paying for Col...	https://milvus.io/blog/milvus-tiered-storage-80-less...
3	⚠️Milvus Week: 3200× Memory Re...	https://milvus.io/blog/introducing-aisaq-in-milvus-...
4	⚠️Milvus Week: GPU_CAGRA	https://milvus.io/blog/faster-index-builds-and-scala...
5	Blog: Milvus gets 40K stars	https://milvus.io/blog/milvus-exceeds-40k-github-s...
6	⚠️Milvus Week: Spatial Meets Sem...	https://milvus.io/blog/unlock-geo-vector-search-wit...
7	⚠️Milvus Week: Array of Structs an...	https://milvus.io/blog/unlocking-true-entity-level-r...
8	Blog: Gemini 3 Pro + Milvus: Buildin...	https://milvus.io/blog/gemini-3-pro-milvus-buildin...
9	⚠️Milvus Week: JSON Shredding	https://milvus.io/blog/json-shredding-in-milvus-fas...
10	⚠️Milvus Week: Introducing Embe...	https://milvus.io/blog/data-in-and-data-out-in-milv...
11	Blog: Multimodal RAG Made Simple...	https://milvus.io/blog/multimodal-rag-made-simpl...
12	Blog: Why Reddit chose Milvus	https://milvus.io/blog/choosing-a-vector-database-f...
13	Blog: OpenAgents x Milvus: How to ...	https://milvus.io/blog/openagents-milvus-how-to-b...
14	Blog: Is MCP Already Outdated? Th...	https://milvus.io/blog/is-mcp-already-outdated-the...
15	Partner Blog: Cloudian Deploys Ne...	https://milvus.io/blog/unlocking-8%C3%97-milvus-...
16	Repost: Power high performance R...	https://milvus.io/blog/power-high-performance-rag...
17	Blog: Beyond Context Overload: Ho...	https://milvus.io/blog/beyond-context-overload-ho...
18	Blog: Drag, Drop, and Deploy: How ...	https://milvus.io/blog/drag-drop-and-deploy-how-t...
19	Blog: Understanding IVF Vector Ind...	https://milvus.io/blog/understanding-ivf-vector-ind...
20	Blog: vLLM Semantic Router + Milv...	https://milvus.io/blog/vllm-semantic-router-milvus-...
21	Blog: How OpusSearch Built Exact ...	https://milvus.io/blog/how-opussearch-built-exact-...
22	Blog: Tokenize Smarter, Retrieve B...	https://milvus.io/blog/tokenize-smarter-retrieve-be...
23	Milvus Meets Late Chunking: Smart...	https://milvus.io/blog/smarter-retrieval-for-rag-late...
24	Blog: How to Debug Slow Requests...	https://milvus.io/blog/how-to-debug-slow-requests...
25	Blog: When Context Engineering Is ...	https://milvus.io/blog/when-context-engineering-is...

25 条记录

Repurposed articles

序号	标题	链接	文章概括	是否合适 repurpose	
1	⚠️Milvus Week: Introducing the Milvus Ngram Index: Faster Keyword Matching and LIKE Queries for Agent Workloads	https://milvus.io/blog/milvus-ngram-index-faster-keyword-matching-and-like-queries-for-agent-workloads.md	介绍 N-gram（n 元）索引用于加速子串/关键词匹配（例如 SQL 的 LIKE / contains），适合在日志、代理记忆、代码片段等场景做“半结构化文本”的快速过滤；并展示如何建索引与查询，以及相对扫描的性能收益。	是	正文大 子串检 程权衡 改写成 & 怎么 substr Reddit
2	⚠️Milvus Week: Stop Paying for Cold Data: 80% Cost Reduction with On-Demand Hot–Cold Data Loading in Milvus Tiered Storage	https://milvus.io/blog/milvus-tiered-storage-80-less-vector-search-cost-with-on-	讲“热-冷数据分层 + 按需加载”的存储/计算成本优化思路：把不常用数据留在冷存储，需要时再加载到热层参与检索，从而降低常驻内存/算力成本，并解释适用场景与运维注意点。	是	核心思 按需加 衡）是 立的成 题/正文 销数字 反感， 围/取决

		demand-hot–cold-data-loading.md			
3	⚠️Milvus Week: 3200× Memory Reduction for Billion-Scale Search with AISAQ in Milvus	https://milvus.io/blog/introducing-aisaq-in-milvus-billion-scale-vector-search-got-3200-cheaper-on-memory.md	介绍 AISAQ 这一面向超大规模向量检索的内存优化方案，强调在十亿级数据下通过特定索引/压缩/数据布局显著降低内存占用，并对比传统索引的资源开销与适用边界。	否	卖点高赖“32径与产到 Red feature需要重开销的化/压缩工作量痕迹。
4	⚠️Milvus Week: GPU_CAGRA	https://milvus.io/blog/faster-index-builds-and-scalable-queries-with-gpu-cagra-in-milvus.md	讲基于 NVIDIA CAGRA（cuVS）做 GPU 加速的 ANN 索引/检索：覆盖索引构建加速、GPU/CPU 混合策略、扩展到更大数据的查询吞吐与成本考量，并给出配置与调参方向。	是	GPU 向GPU、查询、是 Red题；可neutra贴，最在 Milv献”。

5	Blog: Milvus gets 40K stars	https://milvus.io/blog/milvus-exceeds-40k-github-stars.md	里程碑/社区更新：分享 Milvus GitHub star 数达到 40K，并感谢社区贡献者、回顾发展与社区资源。	否	纯里程 度低且 合 Red 盘、可 触发“
6	⚠️Milvus Week: Spatial Meets Semantic: Unlock Geo-Vector Search with Geometry Fields and RTREE Index in Milvus	https://milvus.io/blog/unlock-geo-vector-search-with-geometry-fields-and-rtree-index-in-milvus.md	介绍“地理空间过滤 + 语义向量检索”的组合：用 geometry 字段表达点/线/面，用 R-Tree 做空间索引以加速范围/相交查询，再与向量相似度结合完成 geo-vector 搜索。	是	Geo + s 遍存在 POI、牛 等）， 知识本 成“如 量检索

7	⚠️Milvus Week: Array of Structs and MAX_SIM	https://milvus.io/blog/unlocking-true-entity-level-retrieval-new-array-of-structs-and-max-sim-capabilities-in-milvus.md	围绕“实体级检索/多字段块级匹配”展开：用 array-of-structs 表达一个实体的多片段/多属性；用 MAX_SIM 聚合多向量匹配得分，提升长文档或多块内容的召回质量。	是	MAX_S 似 late ColBEI 检索方 论；改 场景与 评论里

8	Blog: Gemini 3 Pro + Milvus: Building a More Robust RAG With Advanced Reasoning and Multimodal Power	https://milvus.io/blog/gemini-3-pro-milvus-building-a-more-robust-rag-with-advanced-reasoning-and-multimodal-power.md	教程向：用 Gemini 3 Pro 构建 RAG，涵盖数据准备、embedding、向量库写入、检索与组装上下文、生成回答；同时讨论多模态/更强推理能力对 RAG 的影响与实践注意点。	是	主体是与工程为“Ge的踩坑部分写明声明贡献）
9	⚠️Milvus Week: JSON Shredding	https://milvus.io/blog/json-shredding-in-milvus-faster-json-filtering-with-flexibility.md	讨论 JSON 字段过滤性能痛点：对嵌套 JSON 做路径/扁平化索引与“shredding（拆分）”思路，兼顾灵活 schema 与可索引性，提升过滤阶段效率。	是	“JSO慢、怎扁平化话题；indexi很适合

10	⚠️Milvus Week: Introducing Embedding Function	https://milvus.io/blog/data-in-and-data-out-in-milvus-2-6.md	介绍把 embedding 生成下沉到数据库侧的“Embedding Function”思路：写入时自动生成向量并存储，减少应用端 glue code；并涉及数据导入/导出链路与工程化注意点。	是	“emb 还是库 论点； （减少 可观测 成产品

11	Blog: Multimodal RAG Made Simple: RAG-Anything + Milvus Instead of 20 Separate Tools	https://milvus.io/blog/multimodal-rag-made-simple-rag-anything-milvus-instead-of-20-separate-tools.md	展示用 RAG-Anything 构建多模态 RAG：抽取/统一处理图片、PDF、文本等内容并入库检索，强调减少工具拼装复杂度，给出端到端工作流与示例。	是	多模态数据抽立复用个工具 Reddit 销，需复盘（哪些坑
12	Blog: Why Reddit chose Milvus	https://milvus.io/blog/choosing-a-vector-database-for-ann-search-at-reddit.md	以案例复盘方式讲“在生产里选型向量数据库/ANN 方案”的考量：包括规模、延迟、召回、运维与生态等维度，并说明为什么某些取舍更适合其工作负载。	是	选型复口味；X”很容书。建内容平验”+选标题党

13	Blog: OpenAgents x Milvus: How to Build Smarter Multi-Agent Systems That Share Memory	https://milvus.io/blog/openagents-milvus-how-to-build-smarter-multi-agent-systems-that-share-memory.md	教程向：介绍 OpenAgents 的多智能体协作机制，并演示如何把对话、历史解决方案等写入向量存储作为长期记忆，实现跨会话复用与一致性提升。	是	多智能上下文可把重（toke估），实现选确“我献”。
14	Blog: Is MCP Already Outdated? The Real Reason Anthropic Shipped Skills—and How to Pair Them with Milvus	https://milvus.io/blog/is-mcp-already-outdated-the	讨论 Anthropic 的 Skills 与 MCP 的定位差异：Skills 更像轻量行为/流程规范，MCP 提供真实工具能力；并给出把两者组合用于检索/代码搜索等场景的思路与示	是	前半部解释很后半的以降到含澄清

	to Pair Them with Milvus	outdated-the-real-reason-anthropic-shipped-skills-and-how-to-pair-them-with-milvus.md	索/代码搜索等场景的思路与示例。		态度消合 Red论据”
15	Partner Blog: Cloudian Deploys New NVIDIA RDMA for S3-Compatible Storage: A Game-Changer for AI Storage Performance	https://milvus.io/blog/unlocking-8%C3%97-milvus-performance-with-cloudian-hyperstore-and-nvidia-rdma-for-s3-storage.md	转载合作伙伴内容：介绍 RDMA for S3（直达 GPU 内存的数据通路）如何降低 CPU 开销、提升吞吐，并给出与 Cloudian/NVIDIA 的性能数据与引用。	否	合作伙（大量径），当作营普“RD storag则不建
16	Repost: Power high performance RAG for GenAI with HPE Alletra Storage MP + Milvus	https://milvus.io/blog/power-high-performance-rag-for-genai-with-hpe-	转载合作伙伴内容：从 RAG 对存储吞吐/延迟的要求出发，介绍 HPE Alletra Storage MP X10000 的存储能力，并说明其与向量数据库结合的架构价值。	否	典型厂语气与Reddit法是另与指标件为主

		alletra-storag-mp-milvus.md			
17	Blog: Beyond Context Overload: How Parlant × Milvus Brings Control and Clarity to LLM Agent Behavior	https://milvus.io/blog/beyond-context-overload-how-parlant-milvus-brings-control-and-clarity-to-llm-agent-behavior.md	围绕“agent 行为可控”展开：指出把所有规则塞进 prompt 的问题；介绍 Parlant 的 alignment modeling、guidelines 与监督机制；并展示如何用检索动态加载规则、降低 context overload。	是	主题高 agent 关注；方法论为“规实现之即可。
18	Blog: Drag, Drop, and Deploy: How to Build RAG Workflows with Langflow and Milvus	https://milvus.io/blog/drag-drop-and-deploy-how-to-build-rag-workflows-with-langflow-and-milvus.md	教程向：用 Langflow 的可视化界面搭建 RAG（选择向量库、导入数据、跑通检索与生成），适合快速原型、教学或 demo。	是	“可视验/坑点适合 R成可替免“sta today”

19	Blog: Understanding IVF Vector Index: How It Works and When to Choose It Over HNSW	https://milvus.io/blog/understanding-ivf-vector-index-how-It-works-and-when-to-choose-it-over-hnsw.md	系统讲解 IVF 索引原理（k-means 聚类、nlist/nprobe 等参数）以及与 HNSW 的权衡：构建速度、内存、过滤支持与召回/延迟调参方法。	是	几乎全讲解，帖子或怎么选语句删
20	Blog: vLLM Semantic Router + Milvus: How Semantic Routing and	https://milvus.io/blog/vllm	介绍语义路由（按意图/复杂度在多模型间分流）+ 语义缓存（相似问题直接命中缓存）的架构	是	语义路频话题

	Semantic Routing and Caching Build Scalable AI Systems the Smart Way	blog/vllm-semantic-router-milvus-how-semantic-routing-and-caching-scale-ai-systems-the-smart-way.md	似问题直接命中缓存）的架构，并用 vLLM Semantic Router 的示例说明如何用向量库做分布式语义缓存。		与实测值、回作为一
21	Blog: How OpusSearch Built Exact Matching for Enterprise RAG with Milvus BM25	https://milvus.io/blog/how-opussearch-built-exact-matching-for-enterprise-rag-with-milvus-bm25.md	案例复盘：在语义检索“数字/专有名词/精确短语”失灵时，引入 BM25 + 文本匹配过滤实现精确检索；包含 schema、停用词策略、BM25 参数、查询示例与踩坑总结。	是	“语义用 BM2常 Red程复盘意不要片段/关

22	Blog: Tokenize Smarter, Retrieve Better: A Deep Dive into Milvus Analyzer	https://milvus.io/blog/tokenize-smarter-retrieve-better-a-deep-dive-into-milvus-analyzer-for-full-text-search.md	深挖文本分析/分词器：解释 tokenizer + filter 的组合如何影响 BM25/全文检索效果，包含英文/多语言 analyzer 配置示例、run_analyzer 演示与常见配置策略。	是	tokeni 于中立 多、可 stemm 写为“ 南”非
23	Milvus Meets Late Chunking: Smarter Retrieval for RAG	https://milvus.io/blog/smarter-retrieval-for-rag-late-	介绍 Late Chunking：先对整篇文档做 token-level embedding，再按 span 聚合生成 chunk embedding，减少上下文被切碎导致的召回损失；并给	是	Late ch 可迁移 引发“ 更好” techni

		chunking-with-jina-embeddings-v2-and-milvus.md	出与传统 chunking 的对比与实现示例。		为存储
24	Blog: How to Debug Slow Requests in Milvus	https://milvus.io/blog/how-to-debug-slow-requests-in-milvus.md	运维排障指南：用指标与日志定位慢查询（队列/查询/归并阶段），分析常见根因（负载、过滤表达式、索引选择、磁盘/缓存冷启动、后台任务等）并给出对应处置建议。	是	虽然示法论（指标、用的；了怎么会被当发。

Sanitized Reddit adaptation examples

The original catalog pages are retained as source inventory. The old Reddit examples have been replaced by the current drafting boundary below.

Adaptation boundary

- R1/R2 remain anonymous and use an empty Disclosure.
- I built is allowed only when source-supported and without identity, employer,

contributor/maintainer role, affiliation, customer identity, or project representation.

- Write each title from a first-person singular perspective or as an explicit

question ending in ?; do not use a fully third-person declarative title.

- Do not mention Milvus in the title. Every body must mention Milvus exactly

once using source-supported technical background or data; record the basis in milvus_mention_basis. Phrase the mention indirectly as a category example or peer, such as vector databases like Milvus or Milvus and other databases. Never use direct product placement. Never mention Zilliz.

- R1 and R2 never include outbound links.
- The first sentence must establish a source-supported first-person perspective.

Every body must include at least one grammatical language buffer for a judgment or uncertainty. Missing either fails validation; buffers must not hedge source-supported facts, benchmark setup, or measurements.

- Keep the adaptation focused on the engineering problem and constraints.

Tutorials do not end with questions; Discussions use a soft reply invitation when useful rather than a hard interrogative ending.

- Do not repeat benchmark numbers without the full setup and limitations.

Example: substring-filter bottleneck

Title

How would you index substring filters without making writes too expensive?

Body

My current view is that vector search can be fast while keyword or substring filters quietly dominate the query path. A scan-based LIKE '%...%' condition is simple, but it becomes hard to justify once the dataset and concurrent query count grow. In this design, vector databases like Milvus handle the vector search while the substring path is measured separately.

An n-gram inverted index can narrow the candidate set before evaluating the full pattern. The tradeoff is index size, write amplification, and sensitivity to the chosen n-gram range. It also does not remove the need to validate the final match.

I would probably compare n-grams, a separate text index, and a different storage layout against the same mixed semantic-and-exact query set. The unresolved tradeoff is write amplification versus candidate-scan cost. Would love to hear your thoughts.

Milvus mention basis

The source discusses substring filtering alongside Milvus vector search.

Example: GPU build and CPU serving

Title

Does separating GPU index construction from CPU query serving hold up operationally?

Body

One concern I have is that graph-style ANN indexes can be expensive to build, while keeping GPUs online for bursty query serving can be difficult to justify. One design is to use GPU parallelism during construction, persist the resulting graph in a CPU-friendly format, and serve queries from CPU replicas for vector databases such as Milvus.

The architecture shifts cost toward build time, but it introduces compatibility, artifact distribution, rebuild, and rollback questions. It may also behave differently when updates are frequent rather than batch-oriented.

I would likely evaluate artifact movement, update freshness, CPU memory, and cross-version compatibility separately before treating the split as an operational win. Would love to hear your thoughts.

Milvus mention basis

The source describes GPU index construction and CPU query serving for Milvus.